LAB REPORT

BUILDING AI AGENTS WITH LANGGRAPH

Prepared By:

Imane Samir

Contents

I. Introduction: 2
II. Answers to the conceptual questions: 2
Part I: Roles of LangChain, LangGraph and LLM 2
Part II: nodes, edges and state 2
Evaluation: where did the router fail? 3
III. What was implemented: 3
IV. Results: 4
V. Discussion 4
1. Basic graphs and LLM nodes (Parts II to IV): 5
2. Router with conditional edges (Parts V and VI) 5
3. Tools and the agent's decision (Exercise 8 and Part VIII) 5
4. Memory (Exercise 9) 6
5. Visualisation and evaluation 7
6. Capstone: University Assistant (Part X) 7
7. Optional extension: Wikipedia tool 7
VI. Problems encountered and solutions: 8
VII. Limitations and Perspectives: 9
VIII. Conclusion: 10

I. Introduction:

Large language models can give good answers to questions, but they cannot perform a multi-step process, use external tools or remember the conversation without any help. That's why agent frameworks were created to embed the model into a controlled process. LangGraph, the library based on LangChain, describes this process as a graph – a shared state, nodes which change the state, and edges which decide which node will run next.

This report contains my implementation for the practical lab "Building AI Agents with LangGraph". Its purpose is to learn how to build agents as graphs, from the most basic nodes and edges, to add LLM, conditional routing, tools and memory to the graph, and finally building a multi-agent assistant.

The report answers the questions of the lab (section 2), describes what was implemented and the results of the implementation (sections 3 and 4), and gives analysis of each exercise with screenshots (section 5).

LangGraph agents are developed incrementally in this lab, from one node to a multi-agent University Assistant that involves routing, tools, memory, and error handling. All experiments were carried out in Jupyter Notebook using Windows with Python 3.14, LangGraph 1.2.18 and LangChain 1.4.7. The LLM is connected via OpenRouter (an OpenAI API alternative) and uses nvidia/nemotron-3.5-super-vl:free (a free model).

II. Answers to the conceptual questions:

Part I: Roles of LangChain, LangGraph and LLM

1. **LangChain** is a framework that makes it easy to build applications on top of LLMs. LangChain offers pre-built blocks like model wrappers, prompt templates, tools, output parsers, and chains, allowing the LLM to be plugged into data sources, APIs and functions.
2. **LangGraph** is an extension of the LangChain library to allow stateful multi-hop agent workflows by modeling them as a graph. Nodes represent process steps, edges define the flow path, and conditional edges enable branching/looping of the workflow according to the shared state. It also allows the usage of memory via checkpointing.
3. **LLM** stands for Large Language Model which is the language model that forms the foundation of the agent. It understands the text, reasons for the text and generates the response. Within an agent it determines what it should respond to or which tool it should call but without the agent, it has no memory, tools or control flow.

Part II: nodes, edges and state

1. **Node***:* is a unit of work in the graph and is typically a Python function. It takes in the current state, performs an action (call an LLM, run a tool, perform some transformation),

and returns a dictionary containing the state fields that it wishes to update. In this lab, process message is a node.

2. **Edge***:* is a connection between two nodes that shows the order in which things happen. After node A is done, the process moves to node B. Edges can be set in a fixed way using add-edge. They can be based on conditions using add-conditional-edges. With conditions a function decides which node comes next based on the state.
3. **START:** is the designated entry point for the graph. It is not an actual node. Edges that originate at START inform LangGraph of the starting node.
4. **END***:* is the special terminal point of the graph. When the flow reaches END, execution stops and the final state is returned.
5. ***The state is passed between nodes*** **because** it is the shared memory of the workflow. Each node reads what previous nodes produced (for example the analysis, the category, or the messages) and writes its own result. Nodes stay independent and do not call each other directly, and the full state at the end is the output of the graph.

Evaluation: where did the router fail?

The router did not fail on any of the six test cases, giving an accuracy of six out of six which equals one hundred percent. However, the router's result should be read with caution. The test set is small. Each question clearly fits into one category. Ambiguous questions, such as the question "Write a program to compute the derivative of x^2 " which mixes mathematics and programming or short questions were not tested. A free router at temperature zero can still produce outputs, from run to run. The router's evaluation also uses a check meaning the router accepts a malformed label if that label merely contains the expected word.

There are various improvements that would help to make the route more robust:

1. **Structured output**: use llm.with_structured_output() with a Pydantic model or Literal ["mathematics", "programming", "artificial_intelligence"] – in other words, force the model to produce a valid category rather than free-form text.
2. **Few-shot examples** in the classifier prompt, specifically for ambiguous cases.
3. **Enum validation with fallback**: validate the produced label against the acceptable set and fall back to a safe branch (as was done in the capstone project with the GENERAL/fallback approach).
4. **A more difficult test set** that includes ambiguous and out-of-scope questions, an exact-match test rather than substring match, and multiple iterations to estimate variance.

III. What was implemented:

- Single-node graph with a typed state, then an LLM node, then a two-stage agent where the second node uses the analysis generated by the first node.

- Routing: an LLM node as a classifier writes a class to the state, and then conditional edges route the flow either to mathematics, programming, or AI agent.
- Tools: a calculator tool which is safe (as the lab's eval() version is available just for comparing with and running any possible code), current date/time, and a Wikipedia tool (optional). The agent decides whether to use a tool; a conditional edge sends the flow to the ToolNode and returns the control back to the agent.
- Memory with the add_messages reducer and MemorySaver checkpointer, keying by thread_id.
- Capstone project: the University Assistant with an LLM classifier and routing to five specific agents (administration, courses, general, mathematics, and programming agents) that use the tools, with a fallback node in case of no input or errors.

IV. Results:

The results are in the next table we have through the experiment:

	question	expected	got	correct	seconds
0	What is the derivative of x^2?	mathematics	mathematics	True	7.48
1	Solve 2x + 3 = 11	mathematics	mathematics	True	3.49
2	How do I reverse a list in Python?	programming	programming	True	12.47
3	Write a function to check for palindromes.	programming	programming	True	9.43
4	What is a convolutional neural network?	artificial_intelligence	artificial_intelligence	True	15.38
5	Explain overfitting.	artificial_intelligence	artificial_intelligence	True	16.66

```
Routing accuracy: 6/6 = 100%
```

- **Accuracy of routing:** 6/6 (100%) with 3-17 sec/questions (classifier + answering agent).
- **Use of tools**: "What is 125 * 38?" resulted in the use of calculator and produced the output 4750; "What is the capital of Morocco?" received a direct answer without calling a tool.
- **Memory:** with a checkpointer the agent answers "Ahmed" to "What is my name?" in the same thread. Otherwise, it has no idea.
- **The capstone demonstration:** six questions were correctly routed to COURSES, PROGRAMMING, MATHEMATICS, ADMINISTRATION, GENERAL and EMPTY. The trace for the calculator for (15 + 27) * 3 includes the tool call, the tool results 126 and the resulting answer.
- **The wikipedia tool:** the question about the creator of Python results in two wikipedia_search calls and the resulting answer is based on the summaries of the articles obtained.

V. Discussion

Each exercise confirmed the mechanism it was designed to teach. The notes below explain what the outputs show and what they cost.

1. Basic graphs and LLM nodes (Parts II to IV):

The output of the first graph matched perfectly what we would expect, that is, the state,{'message': 'Hello LangGraph!', 'response': 'Agent received: Hello LangGraph!'}. A node needs only return fields it changed; START and END determine where execution should start and end, respectively.

Replacing the string processing function with an LLM function (Exercise 4) made no difference to the graph itself, only the body of the node. This is the core advantage of graph-based agents.

Regarding the two-node agent (Exercise 5), the answer to the query "What is reinforcement learning?" is extensive and well-organized, with sections, tables, and formulas. The second node utilized the analysis done by the first node, indicating state passing through nodes. But there's a trade-off: Two LLMs instead of one.

2. Router with conditional edges (Parts V and VI)

Clean labels (mathematics, programming, artificial_intelligence) came from the classifier, making it easier for the routing to map them to an agent without further parsing. Specialisation obviously altered the responses accordingly.

Mathematical agent computed the derivative using power rule, the programmer agent provided FastAPI sample code, while Artificial Intelligence provided a response appropriate for higher-level students with formulae included. Graphical illustration of specialisation is presented above, depicting the flow of classifier to ai, math, and programming, including the conditional arrows.

Thus, the style and relevance of the answer have been improved due to specialisation, albeit at the cost of one additional call to an LLM.

3. Tools and the agent's decision (Exercise 8 and Part VIII)

Both calculators give 110 as the answer to 25 * 4 + 10, but the ast one gives an error for __import__('os') as "Invalid expression". It accepts only arithmetic nodes; hence, it prevents the execution of untrusted input.

Figure 1 demonstrates how the agent autonomously chooses when the use of a tool is required. "What is 125 * 38?" leads to using a calculator, where the expression is 125 * 38, a tool message 4750, and finally the answer. "What is the capital of Morocco?" leads to an answer (Rabat) without the use of any tool.

This is made possible by the conditional edge tools_condition and the loop from tools to agent.

[31]:
```
for q in ["What is 125 * 38?", "What is the capital of Morocco?"]:
    result = tool_graph.invoke({"messages": [("user", q)]})
    print("=" * 60)
    print("Q:", q)
    for m in result["messages"]:
        m.pretty_print()
```

```
============================================================
Q: What is 125 * 38?
================================ Human Message =================================

What is 125 * 38?
================================== Ai Message ==================================
Tool Calls:
  calculator (call-c942c19f-b2df-4cab-ad04-7c87230abd41)
 Call ID: call-c942c19f-b2df-4cab-ad04-7c87230abd41
  Args:
    expression: 125 * 38
================================= Tool Message =================================
Name: calculator

4750
================================== Ai Message ==================================

125 × 38 = **4,750**.
============================================================
Q: What is the capital of Morocco?
================================ Human Message =================================

What is the capital of Morocco?
================================== Ai Message ==================================

The capital of Morocco is **Rabat**.
```

4. Memory (Exercise 9)

The Memory module relies completely on the checkpointer and thread_id. In the absence of a checkpointer, the function begins fresh, and the model says that it doesn't know the name of the user.

MemorySaver and the thread ahmed-1 led the agent to respond, "Your name is Ahmed." Another thread, however, someone else was not aware of the name

This can be well understood by looking at the add_messages reducer:

The new messages are added to the old, stored list of messages; hence, the entire dialogue gets passed to the LLM at each instance.

[33]:
```
# 1) No persistence: the agent forgets
no_memory_graph = chat_builder.compile()
no_memory_graph.invoke({"messages": [("user", "My name is Ahmed.")]})
r = no_memory_graph.invoke({"messages": [("user", "What is my name?")]})
print("WITHOUT memory:", r["messages"][-1].content)
```

```
WITHOUT memory: I don't know your name — I don't have access to your personal information or identity. If you'd like to tell me your name, I'd be happ
y to use it in our conversation!
```

[34]:
```
from langgraph.checkpoint.memory import MemorySaver

# 2) With persistence: the agent remembers within the same thread
memory_graph = chat_builder.compile(checkpointer=MemorySaver())
config = {"configurable": {"thread_id": "ahmed-1"}}

r1 = memory_graph.invoke({"messages": [("user", "My name is Ahmed.")]}, config)
print("Agent:", r1["messages"][-1].content)

r2 = memory_graph.invoke({"messages": [("user", "What is my name?")]}, config)
print("Agent:", r2["messages"][-1].content)

# A different thread has its own, separate memory
other = {"configurable": {"thread_id": "someone-else"}}
r3 = memory_graph.invoke({"messages": [("user", "What is my name?")]}, other)
print("Other thread:", r3["messages"][-1].content)
```

```
Agent: Hello, Ahmed! It's nice to meet you. How can I help you today?
Agent: Your name is Ahmed.
Other thread: I don't know your name. I don't have access to your personal information or identity unless you share it with me.

If you'd like me to know your name, feel free to tell me!
```

5. Visualisation and evaluation

The classification was fully correct with 6/6 success (100%) as demonstrated by next figure. Latency is more meaningful here than accuracy.

The mathematics questions took 3.5 and 7.5 seconds, the programming ones 9.4 and 12.5 seconds, and the AI ones 15.4 and 16.7 seconds, resulting in an average of about 10.8 seconds each time. Each time incorporates the call to the classifier and answering agent, so the lengthy responses from the AI and programming agents could be a reasonable explanation for the difference. The ratio of the two calls wasn't calculated.

The generated graphs verify the desired behavior: fan-out from the classifier in the router_graph, and a loop between agent and tools in tool_graph.

	question	expected	got	correct	seconds
0	What is the derivative of x^2?	mathematics	mathematics	True	7.48
1	Solve 2x + 3 = 11	mathematics	mathematics	True	3.49
2	How do I reverse a list in Python?	programming	programming	True	12.47
3	Write a function to check for palindromes.	programming	programming	True	9.43
4	What is a convolutional neural network?	artificial_intelligence	artificial_intelligence	True	15.38
5	Explain overfitting.	artificial_intelligence	artificial_intelligence	True	16.66

```
Routing accuracy: 6/6 = 100%
```

6. Capstone: University Assistant (Part X)

The capstone implemented all previous notions and was able to cope with all the six demo inputs: the classifier forwarded them to COURSES, PROGRAMMING, MATHEMATICS, ADMINISTRATION, GENERAL and EMPTY while the date question was given the current date.

125 * 38 + 17 equalled 4,767 and the trace of (15 + 27) * 3 included the calculator function call and the output 126. The memory question was given a "Your name is Ahmed" answer.

The agents responsible for the courses and administration provided helpful general procedures though they noted that they have no access to the policies of the university. It is the most evident restriction of the current design: without the knowledge base, they are unable to give specific replies.

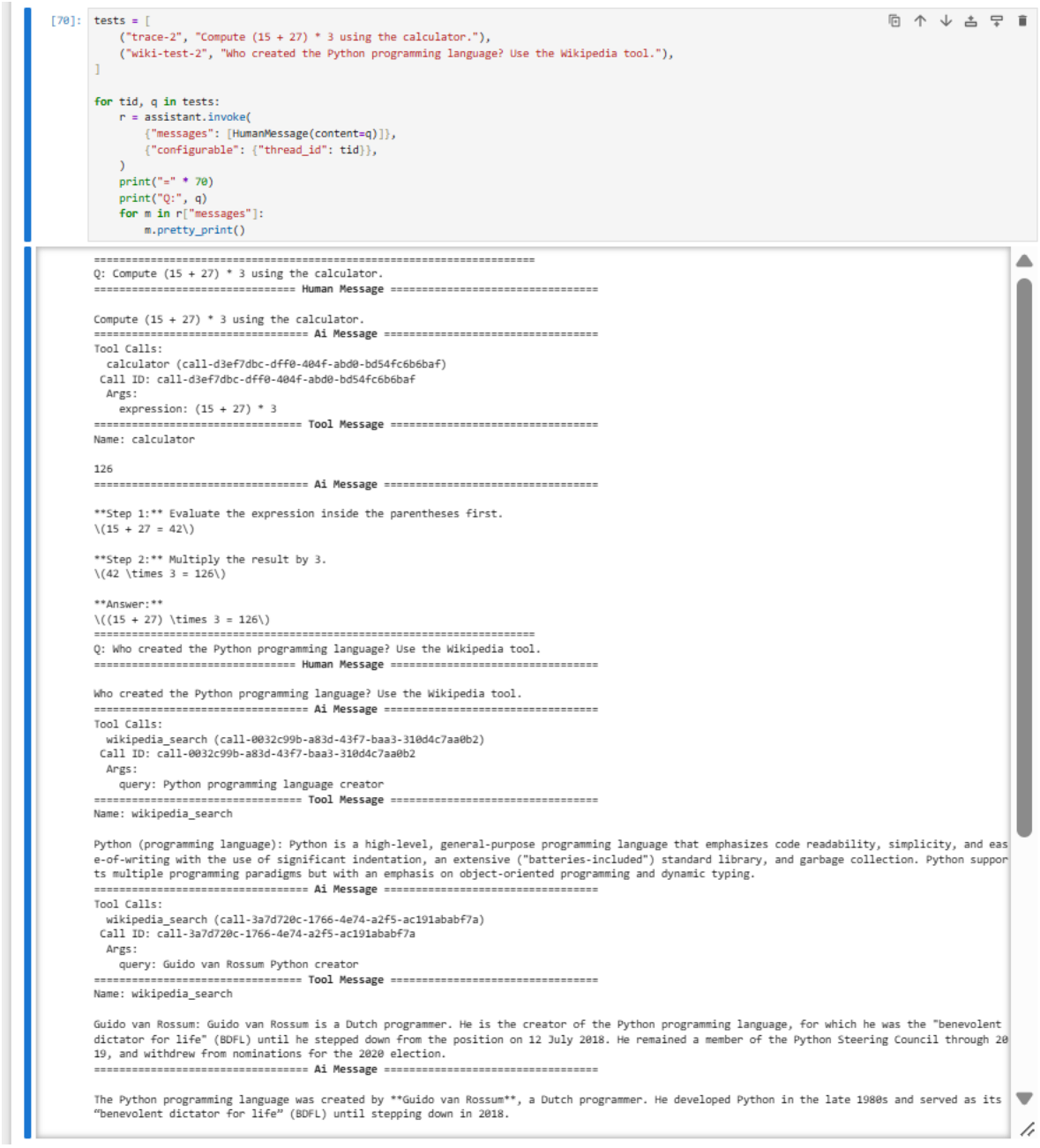
With the daily quota finished, the classifier identified the 429 error while the ERROR category was set up, and the fallback node provided an appropriate response politely. The graph never failed.

7. Optional extension: Wikipedia tool

The following figure presents the final attempt where the agent uses **wikipedia_search** twice to read the summaries and gives the answer that Guido van Rossum invented Python and was its "benevolent dictator for life" up till 2018.

The previous one, constructed on **LangChain's** wrapper, gave the same accurate answer, but every call resulted in **JSONDecodeError**. The answer was given because of the model's memory, and only the message trace showed this.

An accurate final answer does not imply that the tool has been used successfully. The messages must be reviewed, and the retry limit must be set.



```
[70]: tests = [
          ("trace-2", "Compute (15 + 27) * 3 using the calculator."),
          ("wiki-test-2", "Who created the Python programming language? Use the Wikipedia tool."),
      ]

      for tid, q in tests:
          r = assistant.invoke(
              {"messages": [HumanMessage(content=q)]},
              {"configurable": {"thread_id": tid}},
          )
          print("=" * 70)
          print("Q:", q)
          for m in r["messages"]:
              m.pretty_print()
```

```
======================================================================
Q: Compute (15 + 27) * 3 using the calculator.
================================ Human Message =================================

Compute (15 + 27) * 3 using the calculator.
================================== Ai Message ==================================
Tool Calls:
  calculator (call-d3ef7dbc-dff0-404f-abd0-bd54fc6b6baf)
 Call ID: call-d3ef7dbc-dff0-404f-abd0-bd54fc6b6baf
  Args:
    expression: (15 + 27) * 3
================================= Tool Message =================================
Name: calculator

126
================================== Ai Message ==================================

**Step 1:** Evaluate the expression inside the parentheses first.
\(15 + 27 = 42\)

**Step 2:** Multiply the result by 3.
\(42 \times 3 = 126\)

**Answer:**
\((15 + 27) \times 3 = 126\)
======================================================================
Q: Who created the Python programming language? Use the Wikipedia tool.
================================ Human Message =================================

Who created the Python programming language? Use the Wikipedia tool.
================================== Ai Message ==================================
Tool Calls:
  wikipedia_search (call-0032c99b-a83d-43f7-baa3-310d4c7aa0b2)
 Call ID: call-0032c99b-a83d-43f7-baa3-310d4c7aa0b2
  Args:
    query: Python programming language creator
================================= Tool Message =================================
Name: wikipedia_search

Python (programming language): Python is a high-level, general-purpose programming language that emphasizes code readability, simplicity, and eas
e-of-writing with the use of significant indentation, an extensive ("batteries-included") standard library, and garbage collection. Python suppor
ts multiple programming paradigms but with an emphasis on object-oriented programming and dynamic typing.
================================== Ai Message ==================================
Tool Calls:
  wikipedia_search (call-3a7d720c-1766-4e74-a2f5-ac191ababf7a)
 Call ID: call-3a7d720c-1766-4e74-a2f5-ac191ababf7a
  Args:
    query: Guido van Rossum Python creator
================================= Tool Message =================================
Name: wikipedia_search

Guido van Rossum: Guido van Rossum is a Dutch programmer. He is the creator of the Python programming language, for which he was the "benevolent
dictator for life" (BDFL) until he stepped down from the position on 12 July 2018. He remained a member of the Python Steering Council through 20
19, and withdrew from nominations for the 2020 election.
================================== Ai Message ==================================

The Python programming language was created by **Guido van Rossum**, a Dutch programmer. He developed Python in the late 1980s and served as its
“benevolent dictator for life” (BDFL) until stepping down in 2018.
```

VI. Problems encountered and solutions:

The first problem was the LLM provider. Instead of using the OpenAI API, I used OpenRouter. However, the name of the first free model returned 404 as it was no longer available. I made a list of free models that can do tool calling, tested them and selected nvidia/nemotron-3.5-super-vl:free which works with tool calling properly.

The second challenge was that there is a daily request limit for free models (HTTP 429). When it was reached, the capstone's error handling successfully handled the issue: the classifier returned an error category and the assistant responded with a polite message rather than crashing.

The most instructive problem appeared in the Wikipedia tool. LangChain's WikipediaQueryRun always returned JSONDecodeError, as it uses the package that sends a fixed User-Agent, which gets rate-limited by Wikimedia, so the response is always empty. The graph worked, as the tool error is returned to the agent, but the agent called the tool repeatedly around ten times and then responded with its knowledge. It consumed unnecessary requests and returned an ungrounded answer, so I replaced the tool with the custom one that makes a direct call to the Wikipedia API with a timeout and a proper User-Agent.

There are two more minor issues that should be mentioned. Once a new tool gets added, it will be necessary to rebuild the graph since the ToolNode holds the list of tools it was initialized with; the first attempt to test the new tool resulted in “not a valid tool” until the assembling of the graph cell was done. Lastly, the calculator implemented in the lab description fuses eval() which is insecure, so it has been replaced with ast and only takes numbers, and __import__('os') yields "Invalid expression".

VII. Limitations and Perspectives:

The free models are rate-limited and unreliable compared to larger models for the purposes of routing and tool calling. In this case, I observed that my initially free model turned out to become a paid one during the lab. It would be a good idea to introduce either recursion_limit or retry limit in the state. MemorySaver keeps all messages stored in the memory, hence we would need something like a checkpointer (e.g. SQLite) to persist in conversations across multiple runs.

The evaluation is very small and clear since it only evaluates routing capability with the help of six simple questions. Consequently, we have not checked the ability to properly handle tool calls, generate correct answers, calculate costs or handle ambiguous inputs. Additionally, our classifier is a one-shot call to LLM without a confidence score.

However, there are limitations to the functionality of this design as well. There is no knowledge base in courses and the administrative agents, that is why it is impossible to provide answers to the questions on a particular university; the use of the retrieval process or the database such as SQLite with course data can solve this problem. These tools are limited: the calculator calculates math expressions only and the Wikipedia tool provides just one summary and relies on API and its User-Agent regulations. Finally, the average time required to generate evaluations was 10.8 seconds per question.

VIII. Conclusion:

LangGraph provides structure for an LLM app as workflow: typed state, nodes that update it and edges that control it either fixed or conditional. Each exercise added one ability (LLM node, routing, tools, memory, evaluation). The capstone University Assistant combines them: it routes all six demos correctly, utilizes tools when appropriate, remembers a discussion and gracefully fails.

There were practical observations too. Message traces are necessary since they demonstrate what tool calls occurred and the Wikipedia failure was silent. While free LLMs are sufficient for studying, they have quota constraints that demand defensive programming: error handling, fallback node and retry limits. 100% routing accuracy is positive, but it relies on easy questions (six) and is just proof of concept.

Future directions include structured output for the classifier, knowledge base for the courses and administration agents, checkpointer and evaluation.